

Phase 1: Requirements Engineering (Weeks 1)

Before writing a single line of code, you must define the project formally. This satisfies the **Requirement Analysis and Specification** module of your syllabus.

- **Software Life Cycle Model:** Adopt the **Agile development methodology**. In your documentation, state that you are using Agile to iteratively build the Gateway, then the Dashboard, and finally the Anomaly Engine.
- **The IEEE 830 SRS Document:** You will draft a professional Software Requirements Specification using the exact IEEE standard.
 - **Introduction:** Define the purpose (protecting microservices from DDoS and abuse).
 - **Overall Description:** Outline the user characteristics (Developers and System Admins) and operating environment (Docker containers).
 - **Specific Requirements:** List functional requirements (e.g., "The system shall route HTTP requests," "The system shall block requests exceeding 50 per minute") and non-functional requirements (e.g., "The proxy shall add no more than 20ms of latency").

Phase 2: Architecture & System Design (Week 2)

This phase targets the **Architecture and Design** and **System Design** syllabus outcomes. You will create the Unified Modeling Language (UML) diagrams that you will present to your professor.

- **Architectural Styles Used:**
 - **Broker Pattern:** You will use Redis as a broker to instantly track API tokens across the system and manage rate limits.
 - **Micro-Kernel (Plugin) Pattern:** The core system is a simple HTTP proxy. The Rate Limiter, API Key Validator, and Anomaly Guardian are "plugins" that attach to the core proxy.
 - **Pipes and Filters:** The Anomaly Guardian will use this style. Raw HTTP logs (Filter 1) -> Data Parser (Filter 2) -> Anomaly Detection Algorithm (Filter 3) -> Dashboard Alert (Filter 4).
- **UML Diagrams to Draw:**
 - **Component Diagram:** Showing the Gateway, Redis Cache, Anomaly Engine, and Dashboard UI.

- **Sequence Diagram:** Mapping the exact flow of an HTTP request from a user, through the API key check, the Redis token bucket check, and finally to the backend service.

Phase 3: Building the Backend & Anomaly Engine (Weeks 3–4)

This is where the actual engineering happens.

- **The Reverse Proxy (FastAPI):** You will use Python's **FastAPI** framework. It is incredibly fast and perfect for handling asynchronous HTTP requests. You will write a routing function that takes an incoming request, checks the headers, and forwards it to a dummy backend service.
- **The Token Bucket Algorithm (Redis):** To prevent spam, you will implement a **Token Bucket rate limiter**.
 - **Capacity:** The maximum number of requests allowed instantly (e.g., 10 tokens).
 - **Refill Rate:** How fast tokens are added back (e.g., 2 tokens per second).
 - If a client sends a request and the bucket is empty, ShieldAPI immediately rejects it with a 429 Too Many Requests error.
- **The Anomaly Guardian (Python):** You will write a background Python script that continuously reads the proxy's traffic logs. If it detects that a specific IP address is generating an unusual amount of 404 Not Found or 500 Internal Server Error codes, it flags the IP as a potential security threat.

Phase 4: Component-Based GUI Design (Week 5)

This directly satisfies the **User Interface Design** requirement.

- **Framework:** Use React, Vue, or simple HTML/Tailwind CSS if you want to move quickly.
- **The Aesthetic:** Build a dark-mode, futuristic admin command center.
- **Key Components:**
 - **Live Traffic Graph:** A moving line chart showing requests per second.
 - **API Key Manager:** A table where an admin can click "Generate Key" or "Revoke Key."
 - **Anomaly Feed:** A scrolling text box highlighting suspicious IP addresses flagged by your Python engine.

Phase 5: Industrial-Standard Testing (Week 6)

To ace the **Software Testing** module, you must prove your software is stable using industry tools.

- **White-Box Testing (PyUnit):** You will write unit tests for your backend logic. For example, you will write a PyUnit test specifically to check that your Token Bucket algorithm correctly drains and refills tokens mathematically.
- **Black-Box / Automated UI Testing (Selenium):** You will write a Python Selenium script that opens a Chrome browser, automatically logs into your admin dashboard, clicks the "Generate API Key" button, and verifies that the new key appears on the screen.

Phase 6: Cloud Deployment & Presentation (Weeks 7–8)

To meet the deployment requirement, you will take your finished code off your localhost and put it on the internet.

- **Containerization:** You will write a Dockerfile for your FastAPI backend and your React frontend. This proves you understand modern software packaging.
- **Hosting:** You can deploy the backend Docker container for free on **Google Cloud Run** or **Render**, and host the frontend on **Vercel**. You will use a free **Upstash** account for your cloud Redis database.
- **The Presentation:** On presentation day, you won't just show code. You will open with your IEEE SRS and UML Component diagram. Then, you will open the live URL of your dark-mode dashboard. To blow the class away, you can run a simple Python script on your laptop that fires 500 fake requests at your API Gateway, and the class will watch the live graphs spike, the rate limiter kick in, and the malicious IP get blocked in real-time.